

Estrategia y escenarios de prueba para arquitectura basada en colas de mensajes

Laboratorio Logistics Management

Evaluacion del tema 3 de la guia de exposicion y laboratorio
Fecha de revision 3 de octubre de 2026

Conclusion del diagnostico

El proyecto cumple el nucleo tecnico solicitado para el tema 3. Tiene un productor, RabbitMQ como intermediario, una cola principal, un consumidor, confirmacion del consumo al finalizar correctamente, tres intentos ante errores y una cola de mensajes no procesados. La ejecucion automatizada realizada para esta revision termino con 10 pruebas, 0 fallos y 0 pruebas omitidas.

La entrega academica aun no esta completa por si sola. Antes de exponer se deben guardar capturas o reportes de la ejecucion, preparar la presentacion y mostrar de forma visible la confirmacion del mensaje. Tambien conviene automatizar un flujo completo que llegue hasta un servidor SMTP de laboratorio.

Dictamen: cumple tecnicamente; entrega documental y evidencias en preparacion.

Arquitectura evaluada

El laboratorio agrupa dos aplicaciones Spring Boot independientes. shipment-service recibe solicitudes HTTP, guarda el envío en PostgreSQL y publica un evento. RabbitMQ enruta el mensaje hacia shipment.notification.queue. notification-service consume el evento y solicita el envío de correo. Si el procesamiento falla tres veces, el mensaje se rechaza sin reencolar y pasa a shipment.notification.dlq.

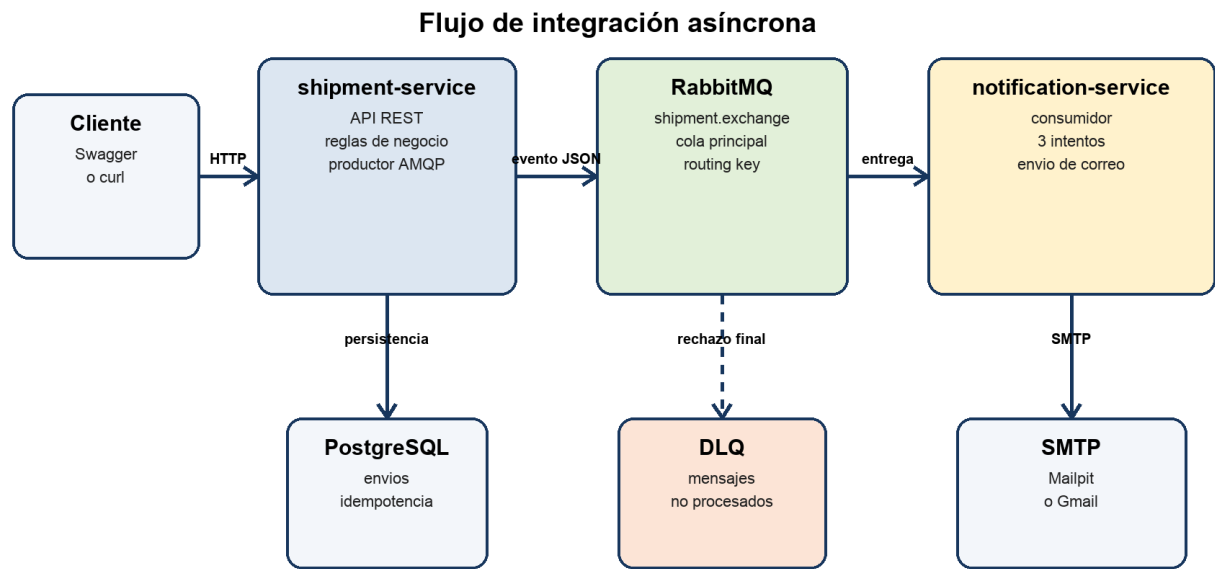


Figura 1 Arquitectura y flujo de mensajes del laboratorio

Verificacion del tema 3

Requisito	Evidencia en el proyecto	Estado	Accion para la exposicion
Productor	RabbitShipmentEventPublisher publica con RabbitTemplate	Cumple	Conservar evidencia del evento publicado
Intermediario	RabbitMQ declarado en docker-compose.yml	Cumple	Mostrar el contenedor saludable
Cola	shipment.notification.queue es durable	Cumple	Mostrarla en RabbitMQ Management
Consumidor	ShipmentEventConsumer usa RabbitListener	Cumple	Relacionar eventId entre productor y consumidor
Envio y recepcion	Evento JSON publicado y convertido a ShipmentEvent	Cumple	Capturar respuesta y logs
Confirmacion	El listener termina normalmente y el contenedor confirma el consumo	Cumple con evidencia pendiente	Mostrar Ready y Unacked en cero tras el exito
Reintentos	Tres intentos totales con esperas de 1 y 2 segundos	Cumple	Capturar los logs intento 1/3 a 3/3
No procesables	RejectAndDontRequeueRecoverer y DLQ	Cumple	Capturar incremento de la DLQ
Automatizacion	JUnit, Mockito y Testcontainers	Cumple	Adjuntar reportes HTML

Aspectos que faltan para completar la entrega

- Presentacion de aproximadamente 20 minutos. No existe una presentacion dentro del repositorio.
- Evidencias exportadas. Los reportes se generan en build, pero deben guardarse capturas o copias para la entrega.
- Prueba automatizada de extremo a extremo hasta Mailpit. Las pruebas actuales validan productor y cola, o consumidor con dependencias simuladas, pero no todo el recorrido en una sola prueba.
- Evidencia directa del ACK. El comportamiento existe por el modo del contenedor, pero debe mostrarse con la cola vacia y sin mensajes Unacked.
- Video corto de respaldo, recomendado por la guia para problemas tecnicos durante la demostracion.
- Como mejora adicional, habilitar publisher confirms si se desea demostrar tambien la confirmacion entre el productor y RabbitMQ.

Estrategia de pruebas

Objetivo

Validar que los componentes intercambian eventos de forma asincrona, que un mensaje correcto se procesa una sola vez y que los errores se controlan mediante validacion, reintentos y DLQ sin detener el consumidor ni producir envios de correo repetidos.

Componentes incluidos

- shipment-service como API, persistencia y productor de mensajes.
- RabbitMQ con exchange, routing key, cola principal, dead letter exchange y DLQ.
- notification-service como consumidor y generador de notificaciones.
- PostgreSQL para persistencia e idempotencia de solicitudes.
- Mailpit como SMTP simulado recomendado, o Gmail con una cuenta propia autorizada.

Puntos de integracion

- HTTP entre el cliente y shipment-service.
- JPA entre shipment-service y PostgreSQL.
- AMQP entre shipment-service, RabbitMQ y notification-service.
- SMTP entre notification-service y el servidor de correo.

Riesgos identificados

ID	Riesgo	Impacto	Tratamiento mediante prueba
R1	El productor no publica o el mensaje no se enruta	Alta	Prueba de publicacion y lectura desde la cola
R2	El consumidor procesa dos veces el mismo eventId	Alta	Prueba de duplicado e idempotencia
R3	Un error genera reintentos infinitos	Alta	Limite de tres intentos y DLQ
R4	Un mensaje invalido detiene el consumidor	Alta	Validacion, rechazo y continuidad

ID	Riesgo	Impacto	Tratamiento mediante prueba
R5	El consumidor no esta disponible	Media	Acumulacion en Ready y consumo al reiniciar
R6	El SMTP no responde	Media	Error controlado, reintento y DLQ
R7	Una solicitud repetida crea dos envios	Alta	Idempotency-Key y restriccion unica

Alcance y exclusiones

El alcance comprende la ejecucion local con Docker Compose, la API de envios, PostgreSQL, RabbitMQ, el consumidor y un SMTP de laboratorio. No incluye carga, alta disponibilidad de RabbitMQ, despliegue en nube, seguridad ofensiva ni pruebas sobre servicios de terceros. Gmail solo se usa con una cuenta propia y datos ficticios.

Ambiente y herramientas

Herramienta	Version o ubicacion	Uso
Java	21	Ejecucion de los microservicios
Spring Boot	4.1.1	API, AMQP, JPA y correo
RabbitMQ	4 management alpine	Broker y consola de administracion
PostgreSQL	17 alpine	Persistencia
Docker Compose	Local	Orquestacion del laboratorio
JUnit y Mockito	Gradle	Pruebas automatizadas
Testcontainers	PostgreSQL y RabbitMQ	Dependencias efimeras de integracion
Swagger	localhost 8080	Ejecucion manual de la API
Mailpit	localhost 8025	Correo simulado y evidencia visual

Datos de prueba

- Correos ficticios como prueba@example.com para escenarios normales con Mailpit.
- fail@email.com para activar el fallo controlado y demostrar los tres intentos.
- Claves de idempotencia unicas por caso, por ejemplo lab-cp01-001.
- Nombres, origenes, destinos y descripciones ficticios, sin informacion personal.

Dependencias reales y simuladas

RabbitMQ y PostgreSQL se ejecutan como dependencias reales en contenedores. Testcontainers crea instancias efimeras para las pruebas de integracion. Mockito simula los puertos de persistencia, publicacion y correo en pruebas de aplicacion. Mailpit simula el servicio SMTP.

Criterios de aprobacion y fallo

- Una prueba se aprueba cuando termina sin excepciones no esperadas y todas sus aserciones coinciden con el resultado esperado.
- Las pruebas de mensajería deben conservar el mismo eventId y shipmentId en productor y consumidor.
- El escenario exitoso debe retirar el mensaje de la cola y registrar la confirmación del consumo.
- El escenario fallido debe ejecutar exactamente tres intentos y terminar en la DLQ, sin requeue infinito.
- Una solicitud o evento duplicado no debe crear un segundo envío ni enviar un segundo correo.
- Cualquier fallo de aserción, mensaje perdido, duplicación o cola en estado inesperado hace fallar el caso.

Matriz de casos de prueba

ID	Escenario	Tipo	Automatizada	Riesgo	Estado
CP-01	Publicacion correcta en RabbitMQ	Integracion	Si	R1	Aprobado
CP-02	Solicitud con correo invalido	Integracion	Si	R4	Aprobado
CP-03	Clave idempotente con datos diferentes	Sistema	Parcial	R7	Parcial
CP-04	Tres intentos y envio a DLQ	Integracion	Si	R3 R6	Aprobado
CP-05	Evento duplicado sin segundo correo	Integracion	Si	R2	Aprobado
CP-06	Consumidor detenido y recuperacion	Sistema	No	R5	Pendiente
CP-07	Flujo completo hasta Mailpit	Sistema	No	R1 R6	Pendiente

Los casos CP-01, CP-02 y CP-04 satisfacen el minimo de automatizacion solicitado por la guia: un escenario exitoso, uno de validacion y uno de falla tecnica.

CP 01 Publicacion correcta en RabbitMQ

Identificador	CP-01
Nombre	Persistencia y publicacion de un envio creado
Tipo de prueba	Integracion
Objetivo	Comprobar que shipment-service guarda el envio y publica un unico evento SHIPMENT_CREATED.
Componentes	CreateShipmentUseCase, PostgreSQL, RabbitShipmentEventPublisher y RabbitMQ.
Precondiciones	Docker disponible. Contenedores efimeros de PostgreSQL y RabbitMQ iniciados por Testcontainers.
Datos de entrada	Idempotency-Key integration-create-1 y datos ficticios de un envio.
Procedimiento	Ejecutar la prueba de integracion de infraestructura. Crear el envio dos veces con la misma clave. Consultar la base y leer la cola.
Resultado esperado	Una fila persistida, un tracking SHP, un solo evento en la cola y ningun segundo mensaje por la repeticion.
Resultado obtenido	La prueba termino aprobada el 3 de octubre de 2026.
Evidencia	shipment-service/build/reports/tests/test/index.html y ShipmentInfrastructureIntegrationTest.
Estado	Aprobado

CP 02 Solicitud con correo invalido

Identificador	CP-02
Nombre	Rechazo de una solicitud HTTP con correo invalido
Tipo de prueba	Integracion
Objetivo	Validar que el controller responde 400 y no invoca el caso de uso cuando el correo no tiene formato valido.
Componentes	ShipmentController, validacion Jakarta y GlobalExceptionHandler.
Precondiciones	Aplicacion de prueba construida con MockMvc.
Datos de entrada	customerEmail invalid y los demas campos obligatorios.
Procedimiento	Ejecutar la prueba automatizada de correo invalido en ShipmentControllerValidationTest.
Resultado esperado	HTTP 400, cuerpo de error y cero interacciones con CreateShipmentPort.
Resultado obtenido	La prueba termino aprobada el 3 de octubre de 2026.
Evidencia	shipment-service/build/reports/tests/test/index.html y ShipmentControllerValidationTest.
Estado	Aprobado

CP 03 Conflicto de idempotencia

Identificador	CP-03
Nombre	Reutilizacion de una clave con datos diferentes
Tipo de prueba	Sistema
Objetivo	Evitar que una misma clave idempotente cree dos envios o acepte datos incompatibles.
Componentes	API de shipment-service, caso de uso de creacion y PostgreSQL.
Precondiciones	Existe un envio creado con la clave lab-cp03-001.
Datos de entrada	Segundo POST con la misma clave y un customerName diferente.
Procedimiento	Crear el primer envio. Repetir el POST cambiando un campo. Consultar la lista de envios.
Resultado esperado	HTTP 409 y una sola fila asociada a la clave. No se publica otro evento.
Resultado obtenido	La regla fue aprobada en ShipmentUseCasesTest. Falta guardar evidencia de la ejecucion HTTP completa.
Evidencia	Prueba de conflicto de idempotencia en ShipmentUseCasesTest y captura pendiente de Swagger.
Estado	Parcial

CP 04 Tres intentos y envio a DLQ

Identificador	CP-04
Nombre	Mensaje que no puede procesarse despues de tres intentos
Tipo de prueba	Integracion
Objetivo	Comprobar que un error controlado no produce un ciclo infinito y que el mensaje termina en la DLQ.
Componentes	RabbitMQ, listener, retry interceptor, ProcessShipmentNotificationUseCase y DLQ.
Precondiciones	RabbitMQ efimero iniciado por Testcontainers.
Datos de entrada	Evento valido con customerEmail fail@email.com.
Procedimiento	Ejecutar la prueba de integracion de retry y DLQ en NotificationRabbitIntegrationTest.
Resultado esperado	Intentos 1/3, 2/3 y 3/3; rechazo sin requeue; mensaje disponible en shipment.notification.dlq.
Resultado obtenido	La prueba termino aprobada el 3 de octubre de 2026 y el mensaje fue leido desde la DLQ.
Evidencia	notification-service/build/reports/tests/test/index.html, NotificationRabbitIntegrationTest y logs del consumidor.
Estado	Aprobado

CP 05 Evento duplicado

Identificador	CP-05
Nombre	Procesamiento idempotente por eventId
Tipo de prueba	Integracion con dependencia SMTP simulada
Objetivo	Comprobar que dos entregas con el mismo eventId generan un solo correo.
Componentes	ProcessShipmentNotificationUseCase y NotificationSenderPort simulado.
Precondiciones	Instancia nueva del caso de uso y sender creado con Mockito.
Datos de entrada	El mismo ShipmentEvent se procesa dos veces.
Procedimiento	Ejecutar la prueba automatizada de evento duplicado en ProcessShipmentNotificationUseCaseTest.
Resultado esperado	NotificationSenderPort recibe una sola invocacion y la segunda entrega se ignora.
Resultado obtenido	La prueba termino aprobada el 3 de octubre de 2026.
Evidencia	notification-service/build/reports/tests/test/index.html y ProcessShipmentNotificationUseCaseTest.
Estado	Aprobado

CP 06 Consumidor detenido y recuperacion

Identificador	CP-06
Nombre	Acumulacion y consumo posterior del mensaje
Tipo de prueba	Sistema
Objetivo	Demostrar que RabbitMQ conserva el mensaje mientras notification-service no esta disponible.
Componentes	shipment-service, RabbitMQ y notification-service.
Precondiciones	Docker Compose activo y cola principal vacia.
Datos de entrada	POST valido con una nueva Idempotency-Key y correo ficticio para Mailpit.
Procedimiento	Detener notification-service. Crear el envio. Verificar un mensaje Ready. Iniciar notification-service y comprobar que Ready vuelve a cero.
Resultado esperado	El productor responde 201, el mensaje permanece en la cola y se procesa cuando vuelve el consumidor.
Resultado obtenido	Procedimiento documentado en README. Ejecucion y captura pendientes para la entrega.
Evidencia	Capturas pendientes de docker compose ps, RabbitMQ Management y logs correlacionados.
Estado	Pendiente

CP 07 Flujo completo hasta Mailpit

Identificador	CP-07
Nombre	Creacion de envio y correo visible en SMTP de laboratorio
Tipo de prueba	Sistema
Objetivo	Demostrar el recorrido desde la solicitud HTTP hasta la notificacion sin usar datos reales.
Componentes	Cliente, shipment-service, PostgreSQL, RabbitMQ, notification-service y Mailpit.
Precondiciones	Compose configurado con MAIL_HOST=mailpit, MAIL_PORT=1025, autenticacion y STARTTLS desactivados.
Datos de entrada	Envio valido con destinatario prueba@example.com y clave lab-cp07-001.
Procedimiento	Crear el envio en Swagger. Seguir eventId y shipmentId en los logs. Abrir localhost:8025 y revisar el mensaje.
Resultado esperado	HTTP 201, evento recibido, ACK, cola vacia y un correo con tracking, origen y destino.
Resultado obtenido	Pendiente de ejecutar con Mailpit y guardar capturas.
Evidencia	Capturas pendientes de Swagger, logs, RabbitMQ Management y bandeja de Mailpit.
Estado	Pendiente

Ejecucion automatizada

La ejecucion del 3 de octubre de 2026 uso el socket de Colima para que Testcontainers pudiera crear PostgreSQL y RabbitMQ efimeros. Los dos proyectos se ejecutaron por separado con su propio wrapper Gradle.

Proyecto	Pruebas	Fallos	Omitidas	Resultado
shipment-service	6	0	0	Aprobado
notification-service	4	0	0	Aprobado
Total	10	0	0	Aprobado

Comandos reproducibles

```
cd shipment-service
DOCKER_HOST=unix:///Users/carolinaecheverri/.colima/default/docker.sock \
TESTCONTAINERS_DOCKER_SOCKET_OVERRIDE=/var/run/docker.sock \
./gradlew clean test
```

```
cd ../notification-service
DOCKER_HOST=unix:///Users/carolinaecheverri/.colima/default/docker.sock \
TESTCONTAINERS_DOCKER_SOCKET_OVERRIDE=/var/run/docker.sock \
./gradlew clean test
```

Si Docker Desktop esta activo, normalmente basta con ejecutar ./gradlew clean test dentro de cada microservicio.

Evidencias que deben guardarse

ID	Evidencia	Situacion
E-01	docker compose ps con todos los servicios activos	Pendiente de captura
E-02	Solicitud y respuesta 201 desde Swagger	Pendiente de captura
E-03	Mismo eventId en productor y consumidor	Disponible en logs
E-04	Cola principal sin Ready ni Unacked despues del exito	Pendiente de captura
E-05	Logs de intento 1/3, 2/3 y 3/3	Disponible en logs
E-06	Mensaje visible en shipment.notification.dlq	Pendiente de captura
E-07	Reportes HTML con 10 pruebas y 0 fallos	Disponible en build/reports
E-08	Correo visible en Mailpit	Pendiente de captura

Las capturas deben usar datos ficticios y mostrar fecha, nombre de la cola, identificadores y resultado. Para conservar los reportes, copiar las carpetas build/reports/tests/test a una carpeta evidencias antes de ejecutar clean.

Guion de demostracion

1. Mostrar la arquitectura y explicar productor, broker, cola, consumidor y DLQ en 3 minutos.
2. Presentar riesgos, estrategia y matriz de casos en 5 minutos.
3. Ejecutar CP-07 como escenario exitoso y mostrar el ACK mediante la cola vacia.
4. Ejecutar CP-04 con fail@email.com y seguir los tres intentos hasta la DLQ.
5. Abrir los reportes automatizados y explicar resultados y limitaciones en 3 minutos.
6. Reservar 1 minuto para preguntas, de acuerdo con la guía.

Instalacion y ejecucion del laboratorio

1. Instalar Docker o Colima y verificar que el daemon este activo.
2. Para la exposicion, configurar Mailpit como SMTP y usar destinatarios ficticios.
3. Desde la raiz del repositorio ejecutar docker compose up -d --build.
4. Abrir Swagger en <http://localhost:8080/swagger-ui/index.html>.
5. Abrir RabbitMQ Management en <http://localhost:15672> con guest y guest.
6. Abrir Mailpit en <http://localhost:8025>.
7. Seguir la ejecucion con docker compose logs -f shipment-service notification-service.
8. Al terminar ejecutar docker compose down. No agregar -v si se desea conservar PostgreSQL.

Analisis de resultados

Los resultados automatizados demuestran que el productor persiste y publica, que la API rechaza datos invalidos y que un mensaje con fallo controlado llega a la DLQ. La prueba de duplicados demuestra que el puerto de correo se invoca una sola vez por eventId durante la vida de la instancia.

La principal limitacion es que la deduplicacion del consumidor reside en memoria. Un reinicio borra los eventId procesados y varias replicas no compartirian el registro. Esto no impide demostrar el tema 3, pero debe explicarse como riesgo y posible mejora mediante una tabla de eventos procesados con restriccion unica.

Otra limitacion es que el test exitoso del productor y el test del consumidor se ejecutan por separado. Una prueba futura debe levantar RabbitMQ y Mailpit, crear el envio por HTTP y esperar hasta encontrar el correo. Esa prueba daria evidencia automatizada de todo el flujo de sistema.

El envio actual usa RabbitTemplate sin publisher confirms. La confirmacion exigida por el tema se puede demostrar como ACK del consumidor. Si el docente pide confirmacion del broker al productor, se deben habilitar publisher confirms y probar los callbacks de ack y nack.

Conclusiones

- La arquitectura implementa productor, cola, consumidor, reintentos y DLQ, por lo que satisface el objetivo tecnico central del tema 3.
- Las 10 pruebas ejecutadas aprobaron y cubren los tres escenarios automatizados obligatorios.

- La calificación dependerá también de evidencias visibles, explicación de los resultados y claridad de la demostración.
- Mailpit es la opción recomendada para el laboratorio porque evita datos personales y dependencias externas.

Lista de entregables

Entregable	Estado	Ubicación o acción
Presentación	Pendiente	Crear diapositivas con el guion de 20 minutos
Diagrama de arquitectura	Incluido	Figura 1 de este documento
Estrategia de pruebas	Incluida	Sección Estrategia de pruebas
Matriz con mínimo cinco casos	Incluida	Siete casos definidos
Código fuente	Disponible	Dos microservicios independientes
Scripts y configuración	Disponible	docker-compose.yml, Dockerfiles y Gradle
Instrucciones	Disponibles	README y este documento
Evidencias ejecutadas	Parcial	Guardar capturas y copiar reportes
Análisis y conclusiones	Incluidos	Secciones finales
Referencias	Incluidas	Sección Referencias

Referencias

- Jaramillo Patiño, Carlos Andres. Guía para la exposición y laboratorio Estrategias de pruebas de integración y de sistema. 2026.
- RabbitMQ. Consumer Acknowledgements and Publisher Confirms. <https://www.rabbitmq.com/docs/confirms>
- RabbitMQ. Dead Letter Exchanges. <https://www.rabbitmq.com/docs/dlx>
- Spring AMQP. Resilience Recovering from Errors and Broker Failures. <https://docs.spring.io/spring-amqp/reference/amqp/resilience-recovering-from-errors-and-broker-failures.html>
- Testcontainers for Java. RabbitMQ module. <https://java.testcontainers.org/modules/rabbitmq/>
- Repositorio Logistics Management. README, código fuente, configuraciones y reportes de pruebas. Revisión del 3 de octubre de 2026.